SVR#

https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html

Description:

Epsilon-Support Vector Regression. The free parameters in the model are C and epsilon. The implementation is based on libsvm. The fit time complexity is more than quadratic with the number of samples which makes it hard to scale to datasets with more than a couple of 10000 samples. For large datasets consider using LinearSVR or SGDRegressor instead, possibly after a Nystroem transformer or other Kernel Approximation. Read more in the User Guide.

Function Signature

```
class sklearn.svm.SVR(*, kernel='rbf', degree=3,
gamma='scale', coef0=0.0, tol=0.001, C=1.0, epsilon=0.1,
shrinking=True, cache_size=200, verbose=False, max_iter=-1)
[source]#

Parameters:

kernel{‘linear’, ‘poly’, ‘rbf’, ‘sigmoid’, ‘precomputed’} or
callable, default=’rbf’

degreeint, default=3

gamma{‘scale’, ‘auto’} or float, default=’scale’
```


```
class sklearn.svm.SVR(*, kernel='rbf', degree=3,
gamma='scale', coef0=0.0, tol=0.001, C=1.0, epsilon=0.1,
shrinking=True, cache_size=200, verbose=False, max_iter=-1)
[source]#

Parameters:

kernel{‘linear’, ‘poly’, ‘rbf’, ‘sigmoid’, ‘precomputed’} or
callable, default=’rbf’

degreeint, default=3

gamma{‘scale’, ‘auto’} or float, default=’scale’
```

```
fit(X, y, sample_weight=None)[source]#

Parameters:

X{array-like, sparse matrix} of shape (n_samples,
n_features) or (n_samples, n_samples)

yarray-like of shape (n_samples,)

sample_weightarray-like of shape (n_samples,), default=None
```

Parameters

`tolfloat, default=1e`

Tolerance for stopping criterion.

`max_iterint, default=`

Hard limit on iterations within solver, or -1 for no limit.

`X{array`

Training vectors, where n_samples is the number of samples and n_features is the number of features. For kernel=”precomputed”, the expected shape of X is (n_samples, n_samples).

`yarray`

Target values (class labels in classification, real numbers in regression).

`sample_weightarray`

Per-sample weights. Rescale C per sample. Higher weights force the classifier to put more emphasis on these points.

paramsdict

Parameter names mapped to their values.

X{array

For kernel="precomputed", the expected shape of X is (n_samples_test, n_samples_train).

Xarray

Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape (n_samples, n_samples_fitted), where n_samples_fitted is the number of samples used in the fitting for the estimator.

yarray

True values for X.

sample_weightarray

Sample weights.

**paramsdict

Estimator parameters.

Code Examples

```
>>> from sklearn.svm import SVR
>>> from sklearn.pipeline import make_pipeline
>>> from sklearn.preprocessing import StandardScaler
>>> import numpy as np
>>> n_samples, n_features = 10, 5
>>> rng = np.random.RandomState(0)
>>> y = rng.randn(n_samples)
>>> X = rng.randn(n_samples, n_features)
>>> regr = make_pipeline(StandardScaler(), SVR(C=1.0,
epsilon=0.2))
>>> regr.fit(X, y)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('svr', SVR(epsilon=0.2))])
```

Notes & Warnings

NOTE

See also NuSVRSupport Vector Machine for regression implemented using libsvm using a parameter to control the number of support vectors. LinearSVRScalable Linear Support Vector Machine for regression implemented using liblinear.

Detailed Documentation

Documentation

Epsilon-Support Vector Regression.

The free parameters in the model are C and epsilon.

The implementation is based on libsvm. The fit time complexity is more than quadratic with the number of samples which makes it hard to scale to datasets with more than a couple of 10000 samples. For large datasets consider using LinearSVR or SGDRegressor instead, possibly after a Nystroem transformer or other Kernel Approximation.

Read more in the User Guide.

Specifies the kernel type to be used in the algorithm. If none is given, ‘rbf’ will be used. If a callable is given it is used to precompute the kernel matrix. For an intuitive visualization of different kernel types see Support Vector Regression (SVR) using linear and non-linear kernels

Degree of the polynomial kernel function (‘poly’). Must be non-negative. Ignored by all other kernels.

Kernel coefficient for ‘rbf’, ‘poly’ and ‘sigmoid’.

if gamma='scale' (default) is passed then it uses 1 / (n_features * X.var()) as value of gamma,

if ‘auto’, uses 1 / n_features

if float, must be non-negative.

Changed in version 0.22: The default value of gamma changed from ‘auto’ to ‘scale’.

Independent term in kernel function. It is only significant in ‘poly’ and ‘sigmoid’.

Tolerance for stopping criterion.

Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared l2. For an intuitive visualization of the effects of scaling the regularization parameter C, see Scaling the regularization parameter for SVCs.

Epsilon in the epsilon-SVR model. It specifies the epsilon-tube within which no penalty is associated in the training loss function with points predicted within a distance epsilon from the actual value. Must be non-negative.

Whether to use the shrinking heuristic. See the User Guide.

Specify the size of the kernel cache (in MB).

Enable verbose output. Note that this setting takes advantage of a per-process runtime setting in libsvm that, if enabled, may not work properly in a multithreaded context.

Hard limit on iterations within solver, or -1 for no limit.

Weights assigned to the features when kernel="linear".

Coefficients of the support vector in the decision function.

0 if correctly fitted, 1 otherwise (will raise warning)

Constants in decision function.

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

Number of iterations run by the optimization routine to fit the model.

Added in version 1.1.

Number of support vectors for each class.

Array dimensions of training vector X.

Indices of support vectors.

Support Vector Machine for regression implemented using libsvm using a parameter to control the number of support vectors.

Scalable Linear Support Vector Machine for regression implemented using liblinear.

LIBSVM: A Library for Support Vector Machines

Platt, John (1999). “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods”

Fit the SVM model according to the given training data.

Training vectors, where n_samples is the number of samples and n_features is the number of features. For kernel=”precomputed”, the expected shape of X is (n_samples,

n_samples).

Target values (class labels in classification, real numbers in regression).

Per-sample weights. Rescale C per sample. Higher weights force the classifier to put more emphasis on these points.

If X and y are not C-ordered and contiguous arrays of np.float64 and X is not a scipy.sparse.csr_matrix, X and/or y may be copied.

If X is a dense array, then the other methods will not support sparse matrices as input.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Perform regression on samples in X.

For an one-class model, +1 (inlier) or -1 (outlier) is returned.

For kernel=”precomputed”, the expected shape of X is (n_samples_test, n_samples_train).

The predicted values.

Return coefficient of determination on test data.

The coefficient of determination, R^2, is defined as $(1 - \frac{u}{v})$, where u is the residual sum of squares ((y_true - y_pred)** 2).sum() and v is the total sum of squares ((y_true - y_true.mean()) ** 2).sum(). The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected value of y, disregarding the input features, would get a R^2 score of 0.0.

Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape (n_samples, n_samples_fitted), where n_samples_fitted is the number of samples used in the fitting for the estimator.

R^2 of self.predict(X) w.r.t. y.

The R^2 score used when calling score on a regressor uses multioutput='uniform_average' from version 0.23 to keep consistent with default value of r2_score. This influences the score method of all the multioutput regressors (except for MultiOutputRegressor).

Configure whether metadata should be requested to be passed to the fit method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with enable_metadata_routing=True (see sklearn.set_config). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to fit if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to fit.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (sklearn.utils.metadata_routing.UNCHANGED) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for sample_weight parameter in fit.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form <component>__<parameter> so that it's possible to update each component of a nested object.

Estimator parameters.

Configure whether metadata should be requested to be passed to the score method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with enable_metadata_routing=True (see sklearn.set_config). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to score if provided. The request is ignored if

metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to score.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this given alias instead of the original name.

The default (sklearn.utils.metadata_routing.UNCHANGED) retains the existing request. This allows you to change the request for some parameters and not others.

Added in version 1.3.

Metadata routing for sample_weight parameter in score.

Comparison of kernel ridge regression and SVR

Support Vector Regression (SVR) using linear and non-linear kernels